CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 05
Augmented Data Structures & Amortized Analysis

How to add new queries to a structure without breaking its guarantees, and what "$O(1)$ amortized" does and does not promise.

Saturday 24 October 2026 · 3 hours

PS2 released today (due Sat 07 Nov) · Project M2 (problem + hypotheses) due today

Two motivating problems

1 · A query the structure doesn't support

A leaderboard stores 10M scores in a balanced BST (from L04's exercise). Product asks for **"what is my rank?"** and **"show ranks 1000–1100"**, with frequent score updates.

An in-order walk is $\Theta(n)$ per query. Rebuilding a sorted array per update is $\Theta(n)$ per update.

2 · A promise that looked fine on paper

An append-only event log uses a growable array: "append is $O(1)$ amortized". In production, p99 append latency is 4 µs, but the **maximum** is 19 ms, and it violates a hard real-time deadline.

Was the $O(1)$ claim false?

BLOCK A · 00:10–01:00

Augmenting data structures

A method, a theorem, and two classic designs

Refresher: what we need from red-black trees

- A BST with colours; height $\leq 2\log_2(n+1) \Rightarrow$ search/insert/delete are $O(\log n)$ **worst case** (R3).
- Insert/delete = a BST operation on one root-to-leaf path + fix-up: $O(\log n)$ recolourings and **≤ 2 rotations (insert), ≤ 3 (delete)**.
- Rotations are the **only** structural changes besides the splice at the bottom.



> For augmentation we only need these three facts: a logarithmic path, $O(1)$ rotations per update, and rotations change only the two nodes' children.

The augmentation method (CLRS 17.2)

1. **Choose an underlying structure** whose guarantees you want to keep (e.g. an RB tree for worst-case $O(\log n)$).
2. **Decide the additional information** to store (per node).
3. **Verify it can be maintained** by every modifying operation (inserts, deletes, *rotations*) without increasing their asymptotic cost.
4. **Develop the new operations** using that information, and prove them correct.

MAINTENANCE THEOREM (CLRS THM 17.1)

Let f be a field in an RB tree such that $x.f$ can be computed in $O(1)$ from x's own data and **the f-fields of its two children only**. Then f can be maintained under insert and delete without changing their $O(\log n)$ cost.

WHY

A change at node x affects only the f values of x's ancestors: $O(\log n)$ of them, each recomputed in $O(1)$. A rotation changes the children of just two nodes: recompute the lower one, then the upper one, in $O(1)$.

■

Order-statistic trees: rank and select in $O(\log n)$

Field: $x.size = x.left.size + x.right.size + 1$ ($nil.size = 0$).

```
1  OS-SELECT(x, i)        ▷ the i-th smallest in x's subtree
2    r = x.left.size + 1  ▷ rank of x within its subtree
3    if i == r  return x
4    if i < r   return OS-SELECT(x.left, i)
5    return OS-SELECT(x.right, i − r)
6
7  OS-RANK(T, x)          ▷ rank of node x in the whole tree
8    r = x.left.size + 1 ; y = x
9    while y ≠ T.root
10     if y == y.p.right
11       r = r + y.p.left.size + 1
12     y = y.p
13   return r
```

OS-RANK INVARIANT

At the start of each iteration, r = the rank of x within the subtree rooted at y.

MAINTENANCE

Moving from y to its parent p: if y is a left child, nothing in p's subtree outside y's precedes x. If y is a right child, all of $p.left$ and p itself precede x: add $p.left.size + 1$. At the root, "rank in y's subtree" is the rank in the tree. ■

Leaderboard: rank $O(\log n)$; ranks $i..i+k$: select i, then k successor steps: $O(\log n + k)$.

Interval trees: a second case study

Store intervals $[lo, hi]$ keyed by lo. Field: $x.max$ = the largest hi in x's subtree (local: a max of three values).

Query: find *some* stored interval overlapping $q = [a, b]$.

```
1 INTERVAL-SEARCH(T, a, b)
2   x = T.root
3   while x ≠ nil and [a,b] does not overlap x.int
4       if x.left ≠ nil and x.left.max ≥ a
5           x = x.left
6       else
7           x = x.right
8   return x
```

WHY GOING LEFT IS SAFE

Going right happens only if $x.left = nil$ or $x.left.max < a$. Then every left interval ends before a and cannot overlap.

Going left with $x.left.max \geq a$: some interval i' in the left subtree has $hi' \geq a$. If none on the left overlaps q, then $lo' > b$ for that i'. Every interval on the right has $lo \geq lo' > b$ (BST order on lo), so none on the right overlaps either. Hence no overlap is lost.

Time $O(\log n)$: one path. ■

Exercise A: design the augmentation

1. Using an order-statistic tree, give an $O(\log n)$ algorithm for COUNT-RANGE(a, b) = the number of keys in $[a, b]$ (a, b need not be stored keys).
2. Which fields can be maintained by the maintenance theorem? Justify each.
   a. the height of x's subtree
   b. the depth of x in the tree
   c. the number of red nodes in x's subtree
   d. the sum over the nodes v of x's subtree of (depth of v below x)
   e. the successor key of x
3. Extend the leaderboard to support "the average score of players ranked $i..j$" in $O(\log n)$.

Pairs · 10 min

BLOCK B · 01:10–02:00

Amortized analysis

Aggregate · accounting · potential · an advanced example

What amortized analysis is, and is not

DEFINITION

Operations have **amortized costs** $\hat{c}_i$ such that for **every** sequence of m operations starting from the initial state,

$$\sum_{i=1}^{m} c_i \leq \sum_{i=1}^{m} \hat{c}_i.$$

- A worst-case bound on **sequences**.
- No probability, no assumption about the inputs.
- A single operation may cost far more than its amortized cost.

LEGACY CORRECTION

The old slides: "Amortized analysis provides a more accurate average-case analysis".

Wrong. Average-case analysis averages over a distribution of inputs. Amortized analysis averages over the operations of one worst-case sequence.

Statement	Guarantee
average case	probability over inputs
expected	probability over our coins
amortized	certain, per sequence

Aggregate method: the binary counter

A k-bit counter, INCREMENT from 0. Cost = the number of bits flipped.

A single INCREMENT can flip all k bits ($0111\cdots1 \to 1000\cdots0$), so the naive bound for n increments is $O(nk)$.

Aggregate: bit i flips every 2^i increments, i.e. $\lfloor n/2^i \rfloor$ times:

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i \ge 0} 2^{-i} = 2n.$$

So the amortized cost is < 2 per increment.

value	bits	flips	total
0	000	–	0
1	001	1	1
2	010	2	3
3	011	1	4
4	100	3	7
5	101	1	8
6	110	2	10
7	111	1	11

Accounting method: dynamic array (doubling)

Append into an array of capacity s. When it is full, allocate $2s$ and copy the s items (cost s), then write (cost 1).

Charge 3 per append:

- 1 pays for writing the item now;
- 1 is stored on the item, to pay for copying *it* at the next resize;
- 1 is stored to pay for copying one *old* item, whose own credit was spent at the previous resize.

Right after a resize to $2s$, the table holds s items and 0 credit. The s appends until the next resize bring in $2s$ credits, exactly enough to copy $2s$ items.

CREDIT INVARIANT

The stored credit is always ≥ 0. That is the only thing an accounting proof must establish: then $\sum c_i \leq \sum \hat{c}_i = 3m$.

Potential method

DEFINITION

A potential Φ maps states to reals. Amortized cost $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$. Telescoping:

$$\sum_{i=1}^{m} \hat{c}_i = \sum_{i=1}^{m} c_i + \Phi(D_m) - \Phi(D_0).$$

If $\Phi(D_i) \geq \Phi(D_0)$ for all i, then $\sum c_i \leq \sum \hat{c}_i$.

Dynamic array (doubling only): $\Phi = 2 \cdot num - size$ (with $\Phi = 0$ for the empty array).

- No resize: $\hat{c} = 1 + 2 = 3$.
- Resize at $num = size = s$: $c = s + 1$; Φ: $2s - s = s \to 2(s+1) - 2s = 2$; $\hat{c} = s + 1 + 2 - s = 3$.

Legacy correction: the condition is $\Phi(D_i) \geq \Phi(D_0)$, not "Φ cannot be negative". (With $\Phi(D_0) = 0$ the two coincide.)

An advanced example: the logarithmic method

Goal: a dictionary supporting INSERT and SEARCH, built only from *static* sorted arrays (useful when the static structure is much simpler or faster).

Bentley–Saxe: keep sorted arrays $A_0, A_1, \ldots$, where A_i is empty or has exactly 2^i items. The non-empty ones follow the binary digits of n.

- **Insert:** a new array of size 1; while two arrays of size 2^i exist, merge them into one of size 2^{i+1} (like the counter's carries).
- **Search:** binary search in each non-empty array: $O(\log^2 n)$ worst case.

CLAIM

Insert costs $O(\log n)$ amortized.

PROOF (AGGREGATE / PER-ELEMENT CHARGING)

A merge into A_{i+1} costs $O(2^{i+1})$, i.e. $O(1)$ per element moved. Each element only moves to higher levels, and there are $\leq \log_2 n + 1$ levels. So over n inserts the total is $O(n \log n)$.

■

Worst single insert: $\Theta(n)$ (a carry chain of all levels). The latency spike again.

Exercise B: amortized reasoning

1. Counter with INCREMENT, where flipping bit i costs 2^i (e.g. higher bits live on slower storage). Is INCREMENT still $O(1)$ amortized? Give the tight amortized bound.
2. Dynamic array with growth factor $r > 1$ (capacity $s \to \lceil rs \rceil$). Give an accounting charge per append and prove it works. What happens as $r \to 1$?
3. Logarithmic method: DELETE is supported by marking items as deleted and rebuilding everything when half the items are marked. Argue that DELETE is $O(\log n)$ amortized.

Pairs · 8 min

BLOCK C · 02:10–02:55

Amortized in practice

Growth factors · memory · latency spikes · de-amortization

Experiment: growth policy vs. copies, memory, latency

Policy ($n = 10^7$ appends)	copies / n	capacity / n	max single append	p99.99
factor 1.25	4.18	1.05	22.9 ms	6.0 µs
factor 1.5	2.39	1.20	17.3 ms	4.5 µs
factor 2	1.68	1.68	18.9 ms	4.4 µs
factor 3	0.72	1.43	10.6 ms	4.7 µs
additive +4096 ($n = 3 \cdot 10^5$)	**36.9**	**1.01**	–	–

8-byte items; per-append latency via `steady_clock` (≈ 2 clock reads per append included); `Student_Resources/dynamic_array_bench.cpp`, g++ 15.2 -O2.

INTERPRET · 5 MIN

1. Check factor 2's 1.68 against theory. Where does $n = 10^7$ fall relative to the last resize?

2. Why does additive growth give 36.9 copies per item?

3. The p99.99 is similar for all factors, but the max is 10–23 ms. What does each statistic tell a real-time engineer?

De-amortization: worst-case $O(1)$ append

Idea: do the copying *incrementally*. When the array of capacity s is half full, allocate the next array (capacity $2s$). Then, on every append:

1. write the new item into **both** arrays (or only the new one, tracking a boundary);
2. copy **2 old items** from the old array to the new one.

By the time the old array is full ($s/2$ more appends), all $s/2$ old items have been copied, and we switch arrays in $O(1)$.

RESULT

Every append is $O(1)$ **worst case** (ignoring allocation cost). The same total work is spread evenly.

COSTS

Two arrays alive (more memory); every operation slightly slower; more complex code (reads must know which array holds each index). Allocation of $2s$ may itself not be $O(1)$ (the OS may zero pages).

The same pattern: incremental rehashing in Redis's dictionaries; incremental garbage collection.

Exercise C: choose and justify

1. An audio-processing thread appends samples to a buffer every 20 µs, and a missed deadline causes an audible click. Which growth strategy do you use, and what guarantee do you need?
2. An order-statistic tree must also report **the median** after every update. What does it cost, and could you do it in $O(1)$ per query by maintaining something extra?
3. A key-value store uses the logarithmic method on disk (an LSM tree). A user complains of occasional 2-second write stalls. Explain the cause in amortized terms, and suggest two mitigations.
4. True or false, with justification: "If every operation of a data structure is $O(1)$ amortized, then any k consecutive operations take $O(k)$ time."

Groups · 12 min

Synthesis

1. **Augment** with fields computable from the node and its children ⇒ maintained in $O(\log n)$, including rotations. Non-local fields (depth, global rank, successor) are not.
2. Order-statistic trees: rank/select $O(\log n)$ worst case; interval trees: overlap queries with a proof of why one path suffices.
3. **Amortized** = a worst-case bound on sequences. Aggregate / accounting (credit ≥ 0) / potential ($\Phi(D_i) \geq \Phi(D_0)$).
4. Amortized bounds hide spikes. When single operations matter, **de-amortize** or pre-allocate.

NEXT WEEK · L06 (SAT 31 OCT)

Dynamic programming as **state-space design**: what must a state remember? The DAG view, evaluation order, reconstruction; LCS, edit distance, matrix-chain multiplication in depth.

Read: CLRS 4e 14.1–14.4; R6. **PS2** (due Sat 07 Nov) Q1–Q2 use today's material.